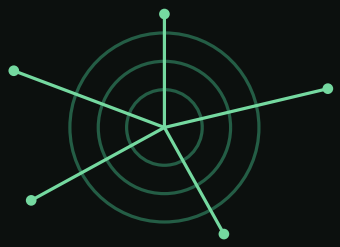
**ARTIFIX AI**

DIAGNOSE • ADVISE • PROTECT • CONNECT

A safer, grounded home-maintenance assistant



# AI Production Enhancement Specification

A coding-agent-ready document consolidating engineering improvements to the existing Artifix AI diagnostic experience. The current four-mode behaviours are treated as implemented and must be preserved.

| DOCUMENT | SCOPE | IMPLEMENTATION PRINCIPLE |
|---|---|---|
| Version 1.0 · 12 October 2026 | Safety, diagnostic quality, grounding, orchestration, matching, evaluation and rollout | Enhance existing code; preserve working flows and avoid unnecessary rewrites. |

## Primary instruction to the coding agent

**Inspect the repository and tests first. Map each recommendation to existing code, identify verified gaps, and make incremental changes only where needed. Preserve all four modes and existing behaviour.**

# Contents


- 1. Purpose and non-goals
- 2. Behavioural contract
- 3. Target architecture and request lifecycle
- 4. Diagnostic reasoning and confidence
- 5. Safety architecture
- 6. Local knowledge and price grounding
- 7. Typed contracts and API boundaries
- 8. Artisan matching and escrow handoff
- 9. Data model and privacy
- 10. Evaluation, testing and observability
- 11. Phased delivery
- 12. Acceptance criteria
- Appendix A. Example response contract
- Appendix B. Architecture and state diagrams
- Appendix C. Repository checklist


## Important scope note

The supplied specification already describes the greeting, diagnosis, emergency, and explicit-artisan-request modes; safety override; matching; BOM display rules; and automated hero-chat-flow tests. This document does not assume these are missing. It describes improvements to verify against the repository and implement only where there is a real gap.

# 1. Purpose and non-goals

Improve reliability, safety, explainability and production operability while retaining the current user-facing behaviours. Treat the original Artifix specification as the baseline contract.

## Behaviours that must not regress

- Mode 0: friendly welcome; category remains null; no fake diagnostic metadata, zero-price estimate, or unsolicited artisan cards.
- Mode 1: explain likely causes and safe next steps; show BOM only when meaningful; do not push artisans unless requested.
- Mode 2: safety-first response and appropriate escalation; safety guidance must not wait for artisan search.
- Mode 3: match the requested craft and area; show eligible verified candidates; create a job/escrow draft only after explicit user action.
- Any newly disclosed hazard overrides a lower-risk mode immediately.
- Keep React/Vite, Express.js, PostgreSQL/Prisma and existing integrations unless repository evidence justifies a change.

## Non-goals

- Do not rebuild the whole product or replace working flows for cosmetic architectural consistency.
- Do not require foundation-model training; begin with orchestration, retrieval, validation and evaluation around the current provider.
- Do not represent remote AI advice as a certified onsite inspection.
- Do not invent live prices, KYC status, artisan availability, response times, dispatch outcomes or payment outcomes.
- Do not initiate payments, share precise location or create a binding booking without required consent and authorization.

**Principle: optimize for the user's informed, safe next step—not maximum chat length or maximum artisan bookings.**

# 2. Behavioural contract

Keep the four visible modes, but internally track intent, safety, evidence, consent and transaction state separately. These dimensions can change independently during a conversation.

| Dimension | Meaning | Engineering rule |
|---|---|---|
| Intent | Greeting, diagnosis, emergency, hiring, follow-up | Re-evaluate each turn; allow transitions and mixed intent. |
| Safety state | No known hazard, possible hazard, high-risk hazard | Independent priority path; ordinary classification cannot suppress it. |
| Evidence | User-observed symptoms, media, answers, contradictions | Separate observations from model-generated hypotheses. |
| DIY eligibility | Whether a task is suitable for a layperson | Use task allowlists, risk rules and explicit stop conditions. |
| Consent | Location, contact sharing, matching, booking | Consent is explicit, scoped and recorded. |
| Transaction state | Advice, draft, booking, funded escrow, milestones | Only trusted backend/payment/chain evidence advances financial state. |

## Conversation quality rules

- Ask the smallest number of high-value clarifying questions needed to distinguish causes or establish safe next steps.
- Use plain Nigerian English by default; mirror Pidgin or another language only when appropriate, without stereotyping.
- Explain what is observed, what is suspected, what remains unknown and what evidence would change the assessment.
- Maintain a concise session summary and update only what changed after each follow-up.
- Do not expose internal chain-of-thought; provide short evidence-based rationale, assumptions and practical next steps.
- Professional assistance may be advised when appropriate, but do not turn ordinary diagnosis into a sales funnel.

# 3. Target architecture and request lifecycle

This is a logical architecture, not a mandate to split the current application into deployed microservices. Implement within current boundaries first; split only when operational needs justify it.

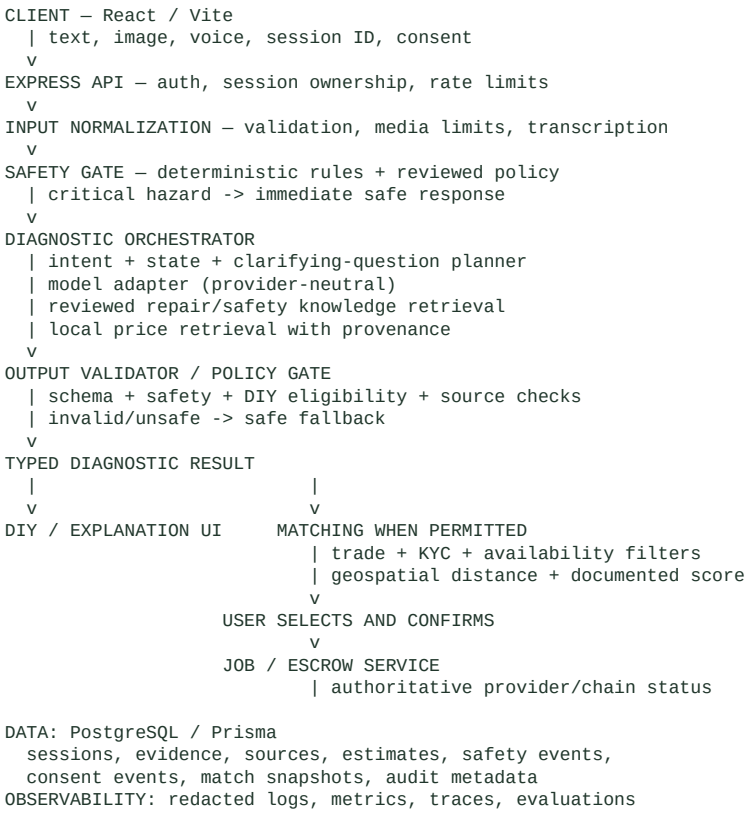



Figure 1. Proposed logical architecture. Safety and output validation are independent of model generation. Matching, booking and financial state remain trusted backend operations.

## Request lifecycle

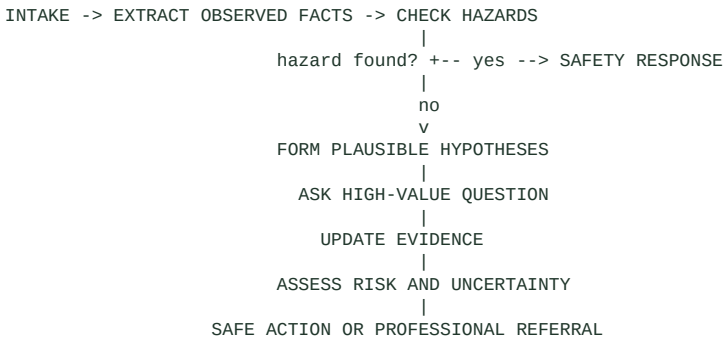
- Authenticate where required; validate session ownership, input size, media type and rate limits.
- Normalize the input; treat user text, images, audio transcripts and retrieved content as untrusted data.
- Run hazard checks before ordinary diagnosis. A critical hazard should not wait for a long model call.
- Resolve session state; separate observed symptoms from inferred causes.
- Retrieve relevant reviewed repair/safety content and location-relevant price records.
- Generate schema-constrained output through the current model adapter.
- Validate schema, safety, DIY policy and evidence provenance. Reject or safely replace invalid output.
- Persist result and audit metadata. Do not stream unvalidated hazardous procedural instructions.
- Run matching only when permitted; verify candidates against authoritative platform records at request time.
- Create job/escrow drafts only after user selection and server-side validation.

## Reliability

Use bounded timeouts/retries for model and retrieval calls. Safety messaging must have a deterministic fallback independent of model-provider availability. Do not cache precise personal data or live availability without suitable controls. Add graceful degradation for provider outages.

# 4. Diagnostic reasoning and confidence

Do not force a single root cause when evidence supports several possibilities. Use a differential diagnosis: short ranked candidate causes, supporting observations, missing evidence and safe questions that discriminate between causes.




## Confidence policy

- Do not treat an LLM self-reported confidenceScore as a calibrated probability.
- Use evidence completeness, contradiction checks, known diagnostic rules, retrieval support and measured validation performance.
- With insufficient evidence, state that multiple causes are possible and ask a targeted question.
- With one plausible but unconfirmed cause, use language such as “one possibility” or “this may indicate.”
- When evidence conflicts, update the hypothesis instead of repeating advice.
- For high-risk equipment or invasive repairs, restrict DIY instructions and recommend qualified inspection.

## Response format

- What I understood: short summary of symptoms.
- What may be happening: plausible causes, marked as unconfirmed where appropriate.
- What you can safely check: actions permitted by the DIY policy.
- When to stop: clear warning signs and stop conditions.
- Estimated cost: only when supported by relevant data; separate materials, labour and extra charges.
- Next step: a question, safe observation or optional verified-artisan route.

**Never invent certainty to sound expert. Prioritize clarity, usefulness and safe action.**

# 5. Safety architecture

Harden the existing SafetyShieldService and hazard override with a versioned taxonomy, deterministic rules, adversarial evaluations and safe fallbacks. The LLM may help interpret context, but must not be the only safety control.

- Detect hazards across misspellings, colloquial language, mixed-language text, transcription errors and indirect descriptions.
- Return immediate safety guidance before nonessential questions, detailed diagnosis or artisan cards.
- Never advise approaching wet electrical panels, touching sparking equipment, handling exposed live wires or entering suspected gas-leak areas to investigate.
- Do not blindly instruct users to operate a breaker or shut a valve if access may be unsafe; guidance must account for the circumstances.
- Give stop/evacuation guidance and direct users to relevant emergency services or utility professionals where appropriate.
- Never claim responders were dispatched unless a real dispatch operation was accepted and status verified.
- Never guarantee arrival time unless the platform has a confirmed commitment.
- Log hazard code, detector/policy version, action and response version while minimizing retained personal data.

## Defence in depth

| Layer | Purpose | Fallback |
|---|---|---|
| Deterministic rules | Catch known high-severity hazards | Conservative safety template |
| Contextual classifier | Interpret context and indirect phrasing | Cautious handling of uncertain high-impact cases |
| Output validator | Block unsafe steps and false status claims | Safe summary/referral |
| Reviewed evaluation set | Measure misses and false alarms | Block releases with critical regression |

## Emergency matching is secondary

Users must receive safe guidance even if location is denied, no artisan is online, the database is degraded or the model provider is unavailable. Location sharing is opt-in and scoped; provide manual location entry.

# 6. Local knowledge and price grounding

Local context is a differentiator only when advice and prices are grounded. Keep the model responsible for explanation and synthesis; keep trusted data records responsible for source, dates and verified facts.

## Knowledge base

- Store reviewed content in retrievable chunks tagged by trade, issue, equipment type, risk level, skill level, locality relevance, review date and version.
- Repair procedures should include prerequisites, allowed user skill, tools/materials, steps, stop conditions, post-repair checks, source, reviewer and version.
- Separate reviewed safety guidance from community tips and anecdotes; lower-trust content cannot override safety rules.
- Filter retrieval by trade, hazard level, approval status and equipment/location before similarity ranking.
- Treat retrieved content as data, not instructions that can override system policy.

## Price record fields

| Field | Purpose |
|---|---|
| Item, category, unit | Consistent identity for part or service |
| Min/max price, currency | Indicative range; NGN where relevant |
| Market/city/area | Avoid treating one city's prices as universal |
| Source type/reference | Supplier quotation, merchant catalogue, verified job or reviewed survey |
| Observed/verified/expiry dates | Suppress stale estimates |
| Sample size/source quality | Describe evidence quality; define semantics |
| Transport/call-out/tax inclusion | Make assumptions visible |

**If no recent location-relevant source exists, say the price is unavailable or provide a broad estimate only if policy permits. Never fabricate a verification date.**

## Cost presentation

- Materials: itemized quantities and sourced unit-price ranges.
- Labour: separate range based on task, locality, complexity and evidence.
- Extra costs: transport, call-out, disposal or emergency surcharge where known.
- Total: range, not a guaranteed quote; label indicative until a provider confirms it.
- Store the artisan's actual quote separately from the AI estimate to evaluate differences.

# 7. Typed contracts and API boundaries

Use one validated internal response contract between orchestration and UI. Adapt names to existing code; do not create duplicate models. Preserve backwards compatibility for existing API consumers.

```
type DiagnosticResponse = {
  sessionId: string;
  intent:
    | "GREETING_OR_GENERAL"
    | "DIAGNOSIS_AND_DIY"
    | "EMERGENCY_HAZARD"
    | "EXPLICIT_ARTISAN_REQUEST";
  categorySlug: string | null;
  safety: {
    level: "none" | "caution" | "high" | "critical";
    hazardCodes: string[];
    immediateActions: string[];
    diyAllowed: boolean;
    escalationRecommended: boolean;
  };
  assessment: {
    symptoms: string[];
    possibleCauses: Array<{
      label: string;
      supportingEvidence: string[];
      missingEvidence: string[];
      likelihoodBand: "low" | "moderate" | "high";
    }>;
    summary: string;
    clarificationQuestion?: string;
    clarificationOptions?: string[];
  };
  guidance: {
    steps: Array<{
      instruction: string;
      risk: "low" | "moderate" | "professional_only";
      stopConditions: string[];
    }>;
    toolsAndMaterials: string[];
  };
  estimate?: {
    currency: "NGN";
    materials: Array<{
      item: string; quantity?: number; unit?: string;
      min: number; max: number; sourceIds: string[];
    }>;
    labourMin?: number;
    labourMax?: number;
    locality?: string;
    observedAt?: string;
    isIndicative: true;
  };
  nextAction:
    | "continue_chat" | "answer_clarification"
    | "offer_professional_help" | "show_emergency_options"
    | "offer_booking";
};
```

Illustrative proposal, not a mandate to replace the current contract. Validate with the repository's existing runtime schema library. Every estimate source ID must resolve to a real record.

## API rules

- Enforce session ownership on every session-specific endpoint.
- Use idempotency keys for job creation and payment initiation.
- Treat client-supplied category, prices, artisan IDs, KYC flags, distances and payment states as untrusted.
- Calculate distance/rank server-side from authoritative records.
- Require explicit authorization for location access, contact sharing and booking.
- Persist schema version and model/prompt/policy versions for migrations and incident review.

# 8. Artisan matching and escrow handoff

Preserve the existing top-three experience for emergency and explicit hiring flows, but make eligibility and availability hard requirements—not soft score contributions.

- Resolve the required trade and competency; exclude cross-trade candidates unless independently verified for the requested skill.
- Filter for required KYC/verification state using authoritative platform data.
- Check service area, current availability/response status, account standing and emergency eligibility.
- Compute distance server-side. Prefer PostGIS/indexed spatial queries if supported; otherwise use a bounded candidate set and tested Haversine implementation.
- Rank eligible candidates using a documented composite score covering skill fit, reliability, ratings, completed jobs, distance and availability.
- Return up to three. If fewer qualify, show fewer; never pad with ineligible candidates.
- Record a match snapshot and ranking reasons for audits and disputes.

## Important correction to the source scoring logic

The supplied specification defines a five-pillar score but sorts primarily by ascending distance and only secondarily by score. This makes the composite score largely ineffective. Choose and document one policy: (A) filter by hard eligibility, then rank by weighted composite score; or (B) deliberately prioritize proximity and use score only as a tie-breaker. Validate against outcomes before tuning weights.

Nearest does not always mean best. For emergency jobs, availability and competency may outweigh small distance differences. Label distance approximate when route or location accuracy is unknown.

## Booking and escrow rules

- Convert-to-job creates a draft unless the existing product flow explicitly and securely funds escrow after confirmation.
- Before confirmation, show scope, estimate versus artisan quote, milestones, cancellation terms, dispute route and fees.
- Never claim funds are locked/released/refunded/settled until authoritative payment-provider or chain state confirms it.
- Use server-side idempotency, verified webhooks/chain receipts and explicit transition rules.
- Keep NGN fiat and on-chain asset flows distinct in data and UI.

# 9. Data model and privacy

Inspect the current Prisma schema and migrations first. Add only missing records or fields; do not duplicate AiDiagnosticSession or existing media models.

| Entity | Suggested fields | Notes |
|---|---|---|
| DiagnosticEvidence | sessionId, evidenceType, normalizedFact, sourceTurnId, confidenceBand, createdAt | Separate observation from inference. |
| KnowledgeSource | sourceId, trade, riskLevel, status, version, reviewedAt, expiresAt | Only approved content grounds sensitive advice. |
| PriceObservation | itemId, locality, min/max, currency, sourceType, observedAt, verifiedAt, expiresAt | Support provenance and stale-price suppression. |
| SafetyEvent | sessionId, hazardCodes, detectorVersion, action, responseVersion, createdAt | Restrict access; minimize raw sensitive text. |
| ConsentEvent | user/session, purpose, scope, granted/revoked, timestamp | Track location/contact-sharing consent. |
| MatchSnapshot | candidate IDs, eligibility snapshot, distance, score, algorithmVersion | Supports ranking audit and disputes. |
| ModelRun/AuditEvent | provider/model, prompt/policy version, latency, validation result | Redact secrets; minimize personal data. |

## Privacy and security

- Apply data minimization and a retention schedule to chat history, voice files, photos and precise location.
- Use encryption in transit and existing at-rest controls; restrict staff access by role.
- Never put provider API keys or privileged prompts in frontend code.
- Validate uploads by content, size and supported format; isolate or scan untrusted files where applicable.
- Defend against prompt injection in text, images, transcripts and retrieved documents; retrieved content cannot authorize actions.
- Support deletion/export where required by product policy and applicable law.
- Logs must omit secrets and unnecessary full transcripts or precise coordinates.

# 10. Evaluation, testing and observability

Extend the existing scripts/test-hero-chat-flow.js regression baseline rather than replacing it. Add dedicated safety, diagnostic, grounding, security, matching and failure-mode suites.

| Suite | Coverage |
|---|---|
| Conversation regression | Greeting null category/no cards; diagnosis no unsolicited artisans; explicit request matches; hazard overrides prior mode. |
| Safety red-team | Wet live electrical equipment, sparking sockets, suspected gas leaks, fire/smoke, structural instability, unsafe breaker access, paraphrases and transcript errors. |
| Diagnostic quality | Representative trade cases, differential hypotheses, clarifications, contradictions, uncertainty and safe stop conditions. |
| Grounding/pricing | Valid source IDs, stale-price suppression, locality/date labels, no invented supplier verification. |
| API/security | Session ownership, auth, rate limits, upload validation, injection, leakage, malformed output. |
| Matching | Trade/KYC eligibility, availability, spatial accuracy, fewer-than-three candidates, deterministic ranking, consent. |
| Booking/escrow | No booking without confirmation; idempotency; forged client state rejected; authoritative payment status. |
| Reliability | Model timeout, retrieval failure, DB degradation, provider outage, bounded retries and safe fallback. |

## Evaluation dataset

- Build a reviewed set of realistic Nigerian home-maintenance scenarios across trades, severity, language style and ambiguity.
- Include positive and negative cases for each hazard rule; track false negatives separately from false positives.
- Have qualified technicians review technical correctness and DIY boundaries; safety reviewers assess high-risk responses.
- Evaluate model/prompt/policy changes against a fixed regression set before release.
- Use user corrections and actual quotes as evaluation evidence only with suitable privacy and consent controls.

## Release-blocking failures

**Block a release for any known critical safety regression, fabricated verification/dispatch/payment claim, unauthorized booking/payment, cross-session data exposure, or regression in the four existing modes. Establish numerical thresholds after baseline measurement for hazard recall, unsafe-instruction rate, unsupported-price rate, schema validity, usefulness, latency and fallback rate.**

# 11. Phased delivery

Phases describe suggested implementation order, not an assertion that the features are absent. The coding agent must map tasks to repository state first.

| Phase | Work | Exit criteria |
|---|---|---|
| 0. Discovery | Inspect routes, services, prompts, schema, UI contracts, tests, logs and deployment config; produce gap map. | Existing behaviour and gaps documented; no speculative rewrite. |
| 1. Safety/contracts | Audit hazard rules, add tests, validate response schema and deterministic fallback. | Critical safety and four-mode tests pass. |
| 2. Diagnostic quality | Add evidence tracking, targeted clarifications, uncertainty and contradiction handling where missing. | Reviewed diagnostic set meets agreed quality bar. |
| 3. Grounding | Reviewed knowledge and price provenance; suppress stale/unsupported estimates. | Estimates traceable or explicitly labelled per policy. |
| 4. Matching integrity | Audit eligibility, availability, scoring and match snapshots; correct ranking semantics. | No ineligible matches; deterministic ranking tested. |
| 5. Operations | Redacted telemetry, eval gates, timeouts, fallbacks, privacy/retention and rollback. | Runbook exists; outage tests pass. |
| 6. Controlled rollout | Internal dogfood, limited beta, technician review and monitoring. | Safety/quality metrics within agreed limits. |

## Suggested ownership

- Engineering lead: architecture, contracts, security, rollout and observability.
- AI engineer: orchestration, retrieval, validation, versioning and evaluations.
- Qualified trade reviewers: repair content, DIY boundaries and technical review.
- Operations: supplier prices, artisan availability, KYC, dispatch and dispute feedback.
- Product owner: consent, estimate presentation, transitions and user-facing claims.

# 12. Acceptance criteria

The work is complete only when code and tests demonstrate the following. If the current code already satisfies an item, document evidence instead of rewriting it.

- ■ Repository discovery completed; existing components and gaps mapped to this specification.
- ■ All four user modes still behave as described in the supplied baseline.
- ■ Safety checks return a conservative immediate response without an LLM call.
- ■ New hazards override prior intent; unsafe instructions are blocked by output validation.
- ■ Diagnostic facts and hypotheses are separate; ambiguous cases clarify or communicate uncertainty.
- ■ Confidence is not called a calibrated probability unless measured and documented.
- ■ DIY eligibility is enforced; high-risk tasks cannot receive unrestricted procedures.
- ■ Every sourced estimate has a valid source, locality and freshness metadata; stale records are suppressed.
- ■ Matching filters trade, verification and availability before ranking; ranking policy is explicit and tested.
- ■ Artisan cards and dispatch claims reflect authoritative state.
- ■ Job/escrow actions require explicit intent, server-side validation, idempotency and authoritative status.
- ■ Session ownership, location consent, upload validation, redacted logs and retention are tested.
- ■ Regression, safety, diagnostic, grounding, matching, security and failure-mode tests pass in CI.
- ■ Deployment config, migrations, rollback, monitoring and runbook are documented.

## Required deliverables from the coding agent

- • Repository assessment: existing components, gaps and files to change.
- • Implementation plan in reviewable steps/commits.
- • Code and migrations only where necessary.
- • Automated tests, results and remaining limitations.
- • Updated environment-variable documentation without committed secrets.
- • Operational notes for hazards, pricing freshness, provider outage and rollback.

# Appendix A. Example diagnostic contract

Illustrative non-emergency AC complaint. Not a confirmed diagnosis or current price quote.


```
{
  "intent": "DIAGNOSIS_AND_DIY",
  "categorySlug": "hvac-and-refrigeration",
  "safety": {
    "level": "caution",
    "hazardCodes": [],
    "immediateActions": [],
    "diyAllowed": true,
    "escalationRecommended": false
  },
  "assessment": {
    "symptoms": ["AC runs but room is not cooling"],
    "possibleCauses": [{
      "label": "Restricted airflow",
      "supportingEvidence": [],
      "missingEvidence": ["Filter condition unknown"],
      "likelihoodBand": "moderate"
    }],
    "summary": "Restricted airflow is one possibility; cause is unconfirmed.",
    "clarificationQuestion":
      "Is airflow weak, or normal but warm?"
  },
  "guidance": {
    "steps": [{
      "instruction":
        "If safe and allowed by the manual, visually check the user-accessible filter.",
      "risk": "low",
      "stopConditions": [
        "Do not open electrical compartments or touch internal components."
      ]
    }],
    "toolsAndMaterials": []
  },
  "nextAction": "answer_clarification"
}
```

# Appendix B. Architecture and state diagrams

## B1. Component architecture

```
flowchart TD
  UI[React / Vite] --> API[Express API + session auth]
  API --> IN[Input normalization / media validation]
  IN --> SAFE[Deterministic SafetyShield]
  SAFE -->|Critical hazard| EM[Immediate safe response]
  SAFE -->|No urgent hazard| ORCH[Diagnostic orchestrator]
  ORCH --> RET[Reviewed knowledge retrieval]
  ORCH --> PRICE[Local price records]
  ORCH --> LLM[Model adapter]
  RET --> VAL[Schema + policy validation]
  PRICE --> VAL
  LLM --> VAL
  VAL -->|Invalid or unsafe| FALL[Safe fallback]
  VAL -->|Valid| RESP[Typed diagnostic result]
  RESP --> UI
  RESP --> MATCH[Matching when permitted]
  MATCH --> ELIG[Trade, KYC, availability filters]
  ELIG --> RANK[Distance + documented score]
  RANK --> CONSENT[User selection and consent]
  CONSENT --> JOB[Job / escrow service]
  JOB --> PAY[Authoritative payment/chain status]
  API --> DB[(PostgreSQL / Prisma)]
  ORCH --> OBS[Redacted telemetry and evaluation]
  SAFE --> OBS
  MATCH --> OBS
```

## B2. Conversation state and safety override

```
stateDiagram-v2
  [*] --> GREETING
  GREETING --> DIAGNOSIS: fault described
  GREETING --> MATCH_REQUEST: explicit hire request
  DIAGNOSIS --> CLARIFY: material uncertainty
  CLARIFY --> DIAGNOSIS: new evidence
  DIAGNOSIS --> MATCH_REQUEST: user asks for artisan
  GREETING --> EMERGENCY: hazard detected
  DIAGNOSIS --> EMERGENCY: hazard detected
  CLARIFY --> EMERGENCY: hazard detected
  MATCH_REQUEST --> EMERGENCY: hazard detected
  EMERGENCY --> EMERGENCY: hazard remains
  EMERGENCY --> DIAGNOSIS: safe to continue
  MATCH_REQUEST --> JOB_DRAFT: user selects provider
  JOB_DRAFT --> BOOKED: user confirms + backend validates
  BOOKED --> [*]
```

## B3. Deployment boundaries

```
flowchart LR
  subgraph APP[Existing application]
    API[Express routes]
    ORCH[Diagnostic orchestration]
    SAF[Safety + output policy]
    MATCH[Matching service]
  end
  subgraph DATA[Trusted data]
    PG[(PostgreSQL)]
    KB[Reviewed knowledge]
    PR[Price observations]
  end
  subgraph EXT[External providers]
    MODEL[LLM provider]
    GEO[Maps/geocoding if used]
    PAY[Payment/blockchain provider]
  end
  API --> ORCH
  ORCH --> SAF
  ORCH <--> KB
  ORCH <--> PR
  API <--> PG
  MATCH <--> PG
  ORCH <--> MODEL
  MATCH <--> GEO
  API <--> PAY
```

# Appendix C. Repository implementation checklist

Map these conceptual areas to existing modules; do not create duplicate services merely to match labels.

### Frontend

AiChatModal, Markdown, clarification controls, BOM accordion, emergency card, accessible loading/error states, consent and booking confirmation.

### API

diagnose, session message, clarify, convert-to-job, authentication, authorization, validation, rate limits and idempotency.

### Safety

SafetyShieldService, hazard taxonomy, deterministic fallback, output validator, test fixtures and policy versioning.

### AI

AiDiagnosticService, model adapter, typed output, clarification planner, retrieval, prompt/model versioning and timeouts.

### Knowledge

Reviewed repair documents, source metadata, retrieval filters, content review, expiration and versioning.

### Pricing

Price observations, locality, source records, freshness rules, provenance and actual quote capture.

### Matching

Trade eligibility, KYC source, availability, spatial query, score policy, deterministic ordering and match snapshots.

### Jobs/escrow

Draft/confirmed states, consent, server validation, idempotency, payment/chain confirmation and dispute semantics.

### Data/security

Prisma schema, migrations, session isolation, media validation, retention, deletion/export and secrets.

### Tests/operations

Existing hero-chat-flow regression, safety red-team, diagnostic dataset, grounding, security, failure injection, metrics, alerts and runbook.

### Final instruction

**Make the smallest correct set of changes that closes verified gaps. Preserve implemented Oga John behaviours. Prefer evidence-backed incremental engineering over a wholesale rewrite. If a recommendation conflicts with current code, document the conflict, test evidence and operational risk, then choose the least disruptive safe solution.**

End of specification.